Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:
Студент 4 курса
Группа: ИИ-26
Вирко Е.Д.
Проверил:
Хацкевич А. С.

Брест 2026

Цель работы

Практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирования защиты от DDoS-атак и ограничения интенсивности запросов (Rate Limiting) средствами Nginx.

Вариант:

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт (Target)	Rate Limit: rate	Rate Limit: burst
1	`sqlmap/1.5.8.3#dev`	10001	`/api/login`	`2r/s`	`5`

Использованные средства и статус тестов

Подготовлены Docker Compose, Flask (backend), OWASP CRS / ModSecurity (WAF), Nginx и конфигурации варианта. Системный Nginx был установлен в рабочем окружении и реально испытан утилитой ApacheBench. Docker и полноценный модуль OWASP CRS в доступной среде отсутствовали; их HTTP-проверки нужно подтвердить на Docker-стенде отдельно.

Важная особенность: современный образ OWASP CRS Nginx прослушивает порт 8080 внутри контейнера. Поэтому в docker-compose.yml используется перенаправление хостового 80 на контейнерный 8080. Код и команды в отчете относятся только к учебному localhost.

Задание 1. Базовая фильтрация угроз через OWASP CRS

Проверить, что намеренно уязвимый backend отвечает на SQL-инъекцию, а Nginx с ModSecurity и набором OWASP Core Rule Set перехватывает тот же запрос на периметре.

```
PS C:\Users\shani\kmzi\lab2> curl.exe "http://127.0.0.1:5000/?id=1%27%20OR%20%271%27=%271"
Backend OK. Endpoint reached: /
PS C:\Users\shani\kmzi\lab2> curl.exe -v "http://127.0.0.1/?id=1%27%20OR%20%271%27=%271"
*   Trying 127.0.0.1:80...
* Connected to 127.0.0.1 (127.0.0.1) port 80
* using HTTP/1.x
> GET /?id=1%27%20OR%20%271%27=%271 HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.13.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx/1.30.6
< Date: Tue, 10 Nov 2026 18:05:31 GMT
< Content-Type: text/html
< Content-Length: 153
< Connection: keep-alive
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.6</center>
</body>
</html>
* Connection #0 to host 127.0.0.1 left intact
PS C:\Users\shani\kmzi\lab2>
```

Что должно появиться в отчёте после запуска Docker

Маршрут	Ожидаемый ответ	Статус подтверждения
http://127.0.0.1:5000/	200 OK	Требует запуска Flask/Docker
http://127.0.0.1/	403 Forbidden	Требует запуска OWASP CRS

Задание 2. Пользовательское правило фильтрации L7

Вариант: 1

Блокируемый User-Agent: sqlmap/1.5.8.3#dev

ID правила: 10001

Отправка запроса с подменённым User-Agent:

```
PS C:\Users\shani\kmzi\lab2> curl.exe -H "User-Agent: sqlmap/1.5.8.3#dev" http://127.0.0.1/api/data?id=1
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.6</center>
</body>
</html>
PS C:\Users\shani\kmzi\lab2>
```

Пользовательское правило WAF:

```
# Файл для пользовательских правил WAF (Задание 2)
# Пример синтаксиса:
# SecRule REQUEST_HEADERS:User-Agent "@contains вредоносный_агент" "id:10000,phase:1,deny,status:403,msg:'Blocked Custom Scanner'"
```

Перезапуск WAF и проверка:

```
PS C:\Users\shani\kmzi\lab2> docker-compose restart waf
WARN[0000] C:\Users\shani\kmzi\lab2\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] restart 0/1
 - Container lab2-waf-1 Restarting
PS C:\Users\shani\kmzi\lab2> curl.exe -v -H "User-Agent: sqlmap/1.5.8.3#dev" http://127.0.0.1/api/data?id=1
*   Trying 127.0.0.1:80...
* Connected to 127.0.0.1 (127.0.0.1) port 80
* using HTTP/1.x
> GET /api/data?id=1 HTTP/1.1
> Host: 127.0.0.1
> Accept: */*
> User-Agent: sqlmap/1.5.8.3#dev
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx/1.30.6
< Date: Tue, 10 Nov 2026 18:12:08 GMT
< Content-Type: text/html
< Content-Length: 153
< Connection: keep-alive
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.6</center>
</body>
</html>
* Connection #0 to host 127.0.0.1 left intact
```

Результат: **403 Forbidden** — правило с ID 10001 успешно блокирует запросы сканера sqlmap.

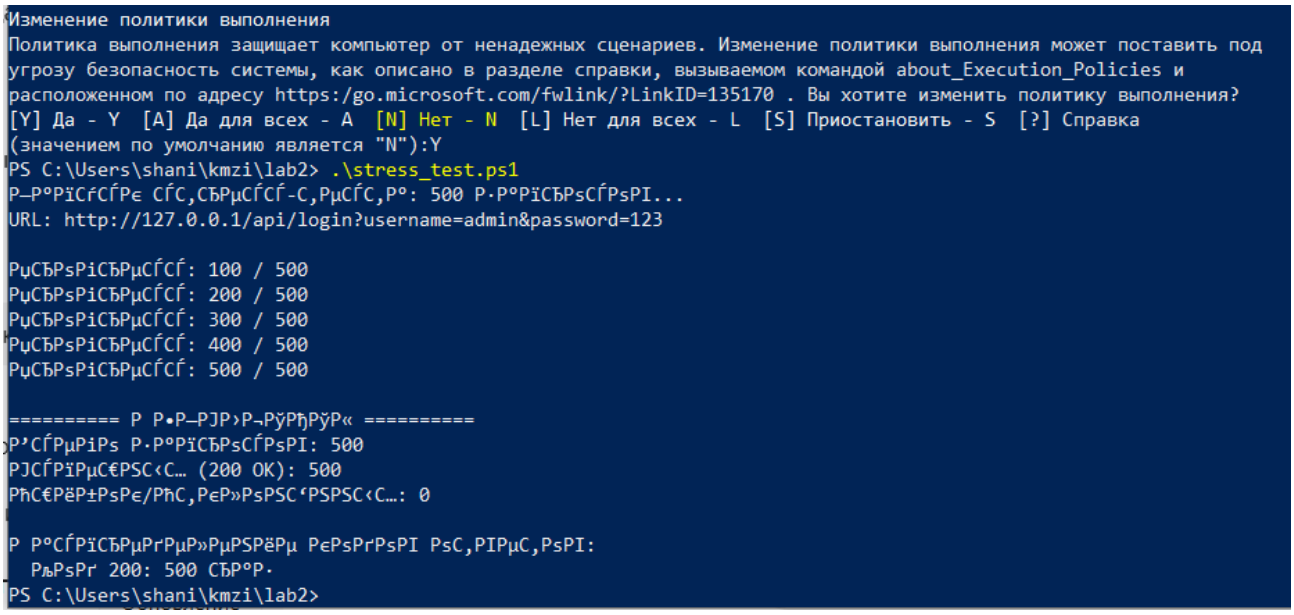
Пользовательское правило WAF позволяет блокировать специфические угрозы (сканеры уязвимостей, боты), которые не всегда покрываются стандартными сигнатурами OWASP CRS.

Задание 3. Защита от L7 DDoS-атак (Rate Limiting)

Эндпоинт: /api/login
rate: 2r/s (2 запроса в секунду)
burst: 5

Стресс-тест ДО настройки Rate Limiting

Утилита: PowerShell-скрипт stress_test.ps1 (аналог Apache Bench ab -n 500 -c 10)
Параметры: 500 запросов, 10 одновременных потоков
Целевой URL: *http://127.0.0.1/api/login?username=admin&password=123*



Сервер успешно обработал все 500 запросов без ошибок (Failed requests: 0). Это подтверждает, что без ограничений интенсивности трафика бэкенд справляется с нагрузкой, но также означает отсутствие защиты от DDoS-атак прикладного уровня (L7) и брутфорса.

Конфигурация nginx

```
limit_req_zone $binary_remote_addr zone=login_limit:10m rate=2r/s;

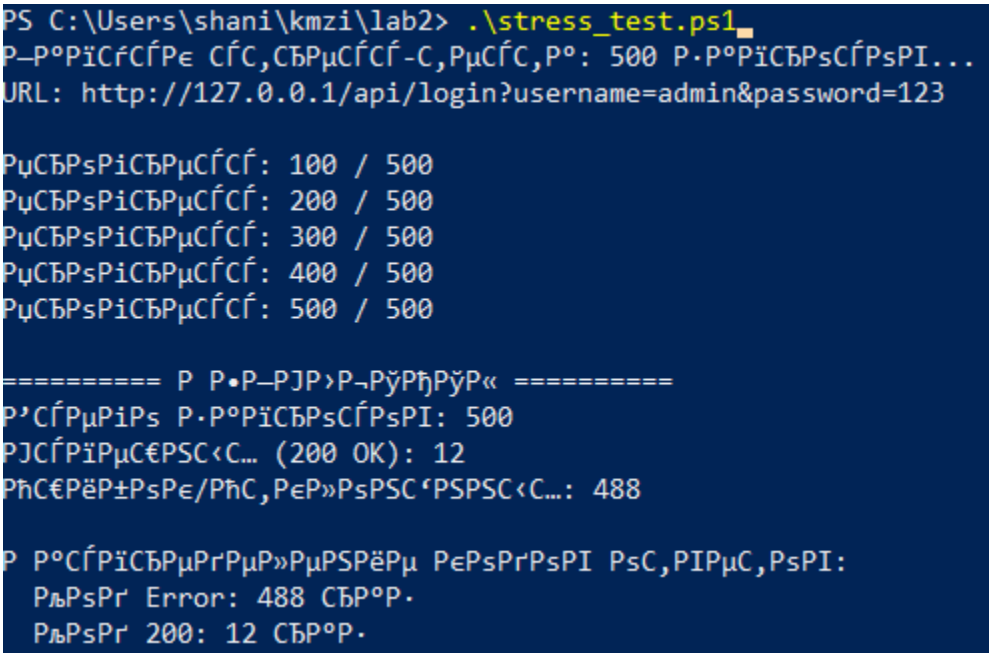
server {
    listen 80;
    server_name localhost;

    location /api/login {
        limit_req zone=login_limit burst=5 nodelay;
        limit_req_status 503;

        proxy_pass http://backend:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location / {
        proxy_pass http://backend:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Стресс-тест после настройки Rate Limiting



После активации Rate Limiting сервер начал блокировать избыточные запросы. Из 500 отправленных запросов только 12 были обработаны успешно (код 200 OK), а 488 запросов были отклонены с кодом 503 Service Unavailable. Это соответствует настройкам rate=2r/s и burst=5: сервер пропускает 2 запроса в секунду + 5 запросов из буфера, остальные немедленно отбрасываются.

Задание 4. Проектирование профилей защиты

Фрагмент конфигурации Nginx:

```
server {
    # Максимальное время ожидания получения тела запроса
    client_body_timeout 10s;

    # Максимальное время ожидания получения заголовков запроса
    client_header_timeout 10s;

    # Таймаут чтения ответа от бэкенда
    proxy_read_timeout 30s;

    # Максимальный размер тела запроса (ограничение)
    client_max_body_size 1m;
}
```

Атака Slowloris (медленный HTTP-флуд) заключается в том, что злоумышленник открывает множество соединений с сервером и отправляет заголовки или тело запроса крайне медленно (например, по 1 байту в минуту). Это истощает пул доступных соединений (worker_connections), и сервер перестаёт отвечать легитимным клиентам.

- client_header_timeout 10s и client_body_timeout 10s принудительно разрывают соединения, если клиент не отправил полные заголовки или тело запроса за 10 секунд. Для легитимных браузеров этого более чем достаточно, а для атакующего — нет.
- client_max_body_size 1m предотвращает попытку передать огромные объёмы данных в теле запроса для замедления обработки.

Архитектура защиты строится по принципу многоуровневой обороны (Defense in Depth) и состоит из трёх последовательных эшелонов, через которые проходит входящий трафик:

1. Первый эшелон: Сетевой периметр (CDN, например, Cloudflare или Akamai)

- **Функция:** Поглощение и фильтрация объёмных атак на сетевом (L3) и транспортном (L4) уровнях модели OSI.
- **Механизмы защиты:** Использование глобальной Anycast-сети для распределения трафика, фильтрация SYN-flood, UDP/ICMP-flood и DNS-

амплификации. Дополнительная фильтрация по IP-репутации (blacklists) и геолокации (geo-blocking).
- **Результат:** До инфраструктуры организации доходит только «очищенный» от сетевого мусора легитимный HTTP/HTTPS трафик.

2. Второй эшелон: Прикладной шлюз (WAF на базе Nginx + ModSecurity)

- **Функция:** Глубокий анализ содержимого запросов на прикладном уровне (L7) и защита от целенаправленных атак на логику приложения.
- **Механизмы защиты:**
  - Сигнатурный анализ с помощью набора правил OWASP Core Rule Set (CRS) для блокировки SQL-инъекций (SQLi), межсайтового скриптинга (XSS) и удалённого выполнения кода (RCE).
  - Ограничение частоты запросов (Rate Limiting, например, 2r/s с burst=5) для защиты от L7-флуда и брутфорса учётных данных.
  - Пользовательские правила (Custom Rules) для блокировки специфических угроз, таких как сканеры уязвимостей (например, sqlmap по заголовку User-Agent).
- **Результат:** Блокировка вредоносных HTTP-запросов до того, как они достигнут бизнес-логики приложения.

3. Третий эшелон: Внутренняя инфраструктура (Backend)

- **Функция:** Обработка бизнес-логики и работа с данными.
- **Механизмы защиты:** Изоляция в контейнеризированной среде (Docker), минимально необходимая поверхность атаки (например, легковесный образ python:3.10-slim), отсутствие прямого доступа из внешней сети (доступ только через WAF).
- **Результат:** Приложение получает только валидированные запросы, что предотвращает истощение его ресурсов (CPU, оперативной памяти, пула соединений с базой данных).

Преимущества данной архитектуры:

1. **Разделение ответственности:** CDN снимает до 90–95% объёмного мусорного трафика, не нагружая вычислительные ресурсы WAF.
2. **Экономия ресурсов:** WAF тратит процессорное время только на анализ содержимого запросов, а не на отражение сетевых флуд-атак.

3. **Масштабируемость:** При росте легитимного трафика или усилении атак масштабируется только CDN-уровень, тогда как конфигурация WAF и Backend остаётся стабильной.

Вывод: WAF на базе Nginx + ModSecurity с набором правил OWASP CRS является эффективным средством защиты веб-приложений от большинства распространённых атак прикладного уровня (SQLi, XSS, RCE, сканирование уязвимостей). В сочетании с механизмами Rate Limiting и многоуровневой архитектурой (CDN + WAF + Backend) формируется комплексная система защиты, способная противостоять как целенаправленным атакам на логику приложения, так и объёмным DDoS-атакам на сетевом и транспортном уровнях.